Remaining Useful Life Prediction via Bayesian Temporal Convolutional Networks

1st Siyi Hong
College of Intelligence and Computing
Tianjin University, China
hsy@tju.edu.cn

2nd Yihan He
School of Mathematics
Tianjin University, China
heyihan@tju.edu.cn

3rd Jianpeng Zhang
National Digital Switching Systems E&T Research Center
Zhengzhou, China
zjp@ndsc.com.cn

4th Chao Jiang
Science and Technology on Particle Transport and Seperation Laboratory
Tianjin, China
jicii@126.com

5th Yingjun Deng
Center for Applied Mathematics
Tianjin University, China
yingjun.deng@tju.edu.cn

2021 International Conference on Advanced Computing and Endogenous Security | 978-1-6654-2449-3/22/$31.00 ©2022 IEEE | DOI: 10.1109/IEEECONF52377.2022.10013353

***Abstract*—In modern industry, the accurate prediction of remaining useful life(RUL) contributes to the equipment safety and economic effectiveness. Aiming at the RUL prediction, this paper proposed a Bayesian temporal convolutional network (BayesianTCN) under the Bayesian deep learning framework. BayesianTCN outputs not only the RUL prediction, but also the associated confidence interval by Monte-Carlo simulation. This quantifies the RUL prediction uncertainty. Experimental results on CMAPSS datasets show that our model has higher fitting degree and lower uncertainty than BayesianLSTM, and performs well whether in simple or complex conditions.**

***Index Terms*—Remaining useful life, Bayesian deep learning, temporal convolutional network, uncertainty quantification, prognosis and health management**

I. INTRODUCTION

In the big-data era, the conventional maintenance strategies like breakdown maintenance and periodic maintenance cannot fully utilize the inspection data, which leads to the trend of predictive maintenance. Predictive Maintenance consists of four components: 1) data acquisition; 2) health indicator (HI) construction; 3) health stage (HS) division; 4) remaining useful life (RUL) prediction [4]. This paper focuses on RUL prediction.

Due to the development of modern machine learning methods, we are step into the era of deep learning and big-data, while conventional machine learning models like support vector machine (SVM) and nearest neighbors still perform well in low-dimensional scenarios.

Deep learning methods can extract features from a large number of monitored data sets autonomously. With the development of sensor technology, the acquisition of condition monitoring data has become easier than ever before. Moreover, the development of big data technology has made it possible to analyze large amounts of data. In the face of a large amount of data, deep learning has a strong feature extraction ability, so the advantage of deep learning in RUL prediction becomes increasingly prominent. Liu *et al.* [5] used temporal convolutional networks (TCN) to do feature learning from the raw vibration signals. Zhang *et al.*[9] developed a hybrid network CNN-LSTM to facilitate the RUL prediction on segmented signals through combining feature extraction, fusion, and regression in a systematic approach. Ma *et al.* [6] designed an algorithm called convolution-based long short-term memory (CLSTM) to achieve RUL prediction of rotating machinery mining the in situ vibration data, which not only took advantage of LSTM, but also incorporating time-frequency features. Chui *et al.*[1] proposed an algorithm which combined recurrent neural network (RNN) with LSTM to predict the RUL of turbofan engines. The former does well in short-term prediction and the latter performs better in long-term prediction.

However, most of the above methods ignore the expression of uncertainty in the RUL prediction and only provide RUL point prediction. The confidence interval of the prediction is always ignored. Nevertheless, when the data contains a lot of noise and the parameters are complex, point prediction will be of no practical significance. When you run multiple predictions over the same dataset, you can see that the RUL is not the same each time. Therefore, the measurement of uncertainty needs to be considered in multi-fault mode, multi-working condition, and multi-noise environment to make the prediction more realistic [8]. Peng *et al.*[7] developed a Bayesian deep-learning-based (BDL-based) method for health prognostics with uncertainty quantification. Hu *et al.* [2] designed a model based on the DBN and diffusion process (DP) to facilitate the prognostic uncertainty. Huang *et al.*[3] proposed a method for aircraft engine based on Bayesian Neural Network(BNN), whose weight are assumed to follow the Gaussian distribution, and they can be updated by Bayes' theorem. In the existing RUL prediction based on deep learning, most of them do not consider the uncertainty in the RUL prediction and cannot give the confidence interval of the prediction. Deep learning based on Bayesian is still in its infancy, and there are few kinds of methods. How to combine with the advantages of

*This work is partially supported by National Natural Science Foundation of China (71701143,62002384), Science and Technology on Particle Transport and Seperation Laboratory, and Tianjin AI Key Project (19ZXZNGX00050) .

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.

other networks to improve the prediction accuracy of RUL and output the confidence interval are the trend of future research. In this article, we propose a RUL prediction model based on BayesianTCN neural network. Compared with the above networks, its weights are represented by probability distributions over possible values. The uncertainty in prediction is quantified by introducing the Bayesian method into convolution layers in TCN structure. The model outputs the posterior distribution of the prediction result and obtains the confidence interval, extending point estimate to interval estimate. Besides, compared with LSTM network, TCN achieves better performance in feature extraction for time series data, thus BayesianTCN has higher prediction accuracy. The experimental results indicate that our model has superior evaluation results than other networks in most cases with the quantification ability of uncertainty.

In the rest of this article, we first give the theoretical background of the model in section II, discussing the principle of variational inference, Bayesian deep learning method, and TCN networks. We describe the proposed method in Section III, and in Section IV, we further elaborate the details of the method through the evaluation of experimental results. Finally, in Section V, we summarize the method and point out the shortcomings.

II. OUR METHOD

A. BayesianTCN structure

In this paper, we propose a novel structure named BayesianTCN (BTCN) based on TCN structure, which outputs the prediction uncertainty while presenting the prediction outcomes, as shown in Fig.1. For RUL prediction problems, TCN can only provide a fixed prediction value as RUL according to given sequence features. While by replacing the convolutional layer in TCN with Bayesian-Conv1d layer, BTCN repeatedly makes prediction over the same sequence through multiple sampling of its parameters, and outputs the standard deviation of results as uncertainty. Therefore, we have three main innovations:

- Propose a whole new structure named BTCN to make predictions over sequence data while calculating the uncertainty of prediction.
- Apply BTCN to solve RUL prediction problems, and provide both point estimation and range estimation of residual life.
- Compared with Bayesian-LSTM structure which is also designed to tackle above problems, BTCN inherits merits of TCN: flexible receptive field size, a faster and more stable converging performance.

Suppose we are given an input sequence $\mathbf{x} = (x_1, \ldots, x_t)$ with a time window length of t, where $x_i (i = 1, \ldots, t)$ is an k-dimensional feature vector that represents the operating status of the machine at each moment. Like convolutional layer in TCN, the Bayesian-Conv1d kernel moves along the sequence direction to extract the features of moment i. Through padding and chomp operation, the output sequence keeps the same length t as the input sequence, so the network produces the mapping

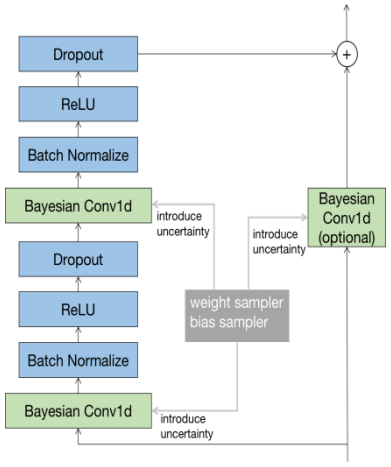


Fig. 1. BayesianTCN

$$\widehat{y_1}, \ldots, \widehat{y_t} = f_{BTCN}(x_1, \ldots, x_t) \quad (1)$$

where $\widehat{y_i}$ represents the characteristic value of the machine's running state at moment i. For the sequence-to-sequence prediction task, $\widehat{y_i}$ is regarded as the RUL and the model outputs the whole vector $(\widehat{y_1}, \ldots, \widehat{y_t})$. And for the prediction task of sequence-to-one, we add a fully connected layer in the shape of $(t, 1)$ after BayesianTCN, so that we get Y_t as the RUL at moment t, calculated as:

$$Y_t = f_{FNN}(\widehat{y_1}, \ldots, \widehat{y_t}) \quad (2)$$

B. Algorithm

The construction idea of BayesianTCN is to introduce the Bayesian method into the convolution layer, which is mainly used in parameter initialization and its optimization process. The following will describe how is uncertainty introduced and how we model its loss to objectively improve the confidence over its prediction and reduce the variance without dropout.

Weight Sampling: Parameters of each Bayesian layer, including the weight $W_{(n)}^{(i)}$ and bias $b_{(n)}^{(i)}$ are initialized in feed-forward operaion using formulae Eq.(3). Here ρ and μ denotes the standard deviation and the mean for the samples linear transformation parameters.

$$\begin{cases} W_{(n)}^{(i)} = \mathcal{N}(0,1) * log\left(1 + \rho^{(i)}\right) + \mu^{(i)}, \\ b_{(n)}^{(i)} = \mathcal{N}(0,1) * log\left(1 + \rho^{(i)}\right) + \mu^{(i)} \end{cases}$$

Loss Calculation: Since the weights and biases of Bayesian layers vary on each feed-forward operation, the network can generate different results. In the n-th sampling, the loss value is calculated as Eq. (4), which is differentiable to all of its parameters. In the formula, $p(w)$ is a low-entropy distribution pdf set by hand and $q(w|\theta)$ is the posterior empirical distribution pdf for our sampled weights.

$$C^{(n)}\left(w^{(n)}, \theta\right) = \left(logq\left(w^{(n)}, \theta\right) - logp\left(w^{(n)}\right)\right) \quad (4)$$

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.

Here we use variational inference (VI) to approximate the posterior distribution and the closeness is measured by KL divergence.

$$KL\big(q(z) \parallel p(z \mid x)\big) = E\big(\log q(z)\big) - E\big(\log p(z,x)\big) + \log p(x) \quad (5)$$

The minimization of KL divergence is converted to the maximization of evidence lower bound (ELBO) because marginal distribution $\log p(x)$ is difficult to solve.

We combine the Monte Carlo method and gradient descent method to find the optimal solution of ELBO to represent average cost $f(w,\theta)$. The reparameterization trick is used to handle the high-dimensional integration over $q(z)$,

Taking the derivative of λ,

$$\nabla ELBO = \int q(z,\lambda)\nabla \log q(z,\lambda)\log\frac{p(x,z)}{q(z,\lambda)}dz \quad (6)$$

Weight Optimization: We can optimize weights and biases by updating the variational parameter $\theta = (\rho, \mu)$ based on formulae Eq.(7) and Eq.(8).

$$\nabla_\mu = \frac{\delta f(w,\theta)}{\delta w} + \frac{\delta f(w,\theta)}{\delta \mu} \quad (7)$$

$$\nabla_\rho = \frac{\delta f(w,\theta)}{\delta w}\frac{\varepsilon}{1+e^\rho} + \frac{\delta f(w,\theta)}{\delta \rho} \quad (8)$$

In the formulae, ε denotes Gaussian distribution N (0, 1), and $f(w,\theta)$ is the mean of $C^{(n)}\big(w^{(n)},\theta\big)$ in s samples.

What is special of our model is the introduction of Monte Carlo method for parameter optimization in BTCN, as illustrated in Algorithm II-B. Each training cycle has N sets of inputs, where **x** is the input sequence of length t and y represent the RUL at the last moment of sequence **x**.

Algorithm 1 Model Training with Monte Carlo method

Input: $(\mathbf{x}^N, y^N) \in D_L$, sampling times as **s**
Output: $\mathbf{w}_t$:parameters in Bayesian-conv1d layers, $\mathbf{w}_l$:parameters in other layers of the model

```
ρ, μ ← initialize standard deviation and mean to generate w_t
w_t ← generate parameters using formulae Eq.(3)
w_l ← initialize other parameters in the network
for (x, y) in (x^N, y^N) do
    for n = 1 → s do
        p(w^(n)) ← y
        for each cell j in hidden or output layers do
            I_j = Σ_i w_j O_i
            O_j = activate(I_j)
        end for
        q(w^(n)|(ρ, μ)) ← O_j in output layer
        cost_n ← compute C^(n)(w^(n), θ) in nth sample
    end for
    f(w, θ) ← 1/s * Σ_{n=1}^{s} cost_n
    for each cell j in hidden layers do
        compute Err_j for each cell using f(w, θ)
        if w_j ∈ w_t then
            update ρ, μ using formulae Eq.(7) and Eq.(8)
            generate w_t in terms of ρ, μ
        else
            optimize w_j directly
        end if
    end for
end for
```

C. Training Setup

The training and prediction process of the BayesianTCN model is as follows.

(1) Data preprocessing. Load the data set for model training and prediction.

- The z-score standardization is performed on each feature vector representing the operating status or condition of the machine, thereby weakening the interference of features of different metrics on modeling.

$$z = \frac{x-\mu}{\sigma} \quad (9)$$

- Set the RUL label. The RUL label calculated according to the running time decreases linearly with time, then the label of engine with life cycle C_{max} is $y_i = C_{max} - i$, where i denotes the running moment. However, in the actual scenario, the machine is considered healthy for an initial period, and after the breaking point, its remaining life decreases. Therefore, the piecewise linear function is used to set the RUL label, as shown in Fig. 2: The RUL value of the phase is set to a constant value $C_{max}/2$, and then the RUL decreases as the number of operating cycles increases. The corresponding formula is as Eq. (10).

$$y_i = \begin{cases} C_{\max}/2, i < C_{\max}/2 \\ C_{\max} - i, i \geq C_{\max}/2 \end{cases} \quad (10)$$

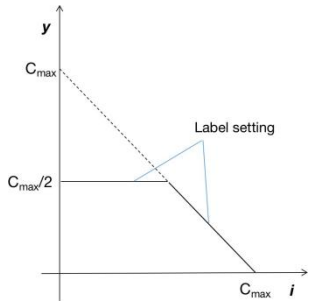


Fig. 2. label setting

- Construct training and test samples through sliding time windows. For the same machine sample, sliding the window from the first data of the original data, moving a one-time unit at a time in the direction of increasing cycle number.

(2) Train the BayesianTCN network model

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.

Given an input sequence $\mathbf{x} = (x_1, \ldots, x_t)$ its correspond- ing label is y_t which represents the RUL at moment t. The goal of our model is to find a network that minimizes loss between the label y_t and the prediction Y_t. Different from ordinary network training, when we update the network parameters, the loss value is the average cost of multiple samples. The specific algorithm is given in Section II-B.

(3) RUL prediction with confidence interval
For each input $\mathbf{x}$, by sampling its output s times, the posterior distribution of s predictions $Y: Y^1, \ldots Y^s$ can be obtained. As it is difficult to get the analytic expression of posterior distribution in practice, here we use the results of multiple predictions to approximately express empirical distribution. Correspondingly, calculate the empirical mean Y_{mean} and empirical standard deviation Y_{std} of the distribution Y, and set $Z_{\alpha/2}$ to be the standard score when the confidence is α, then the confidence interval CI of the prediction result can be expressed as:

$$CI = \left[Y_{mean} - \left(Z_{\alpha/2} * Y_{std}\right), Y_{mean} + \left(Z_{\alpha/2} * Y_{std}\right)\right] \quad (11)$$

Therefore the uncertainty is quantified, and we extend the predicted point estimate to an interval estimate.

III. Empirical Study

A. C-MAPSS dataset description

This paper uses the C-MAPSS dataset provided by NASA as the verification object of the model. This dataset only uses historical data (including from operation to failure) to predict the remaining life of equipment, without considering the underlying physical factors. C-MAPSS data comes from turbofan engine, which is a modern gasoline turbine engine used by NASA Space Exploration Agency. There are four datasets, named FD001, FD002, FD003, and FD004. Each data set is collected under different working conditions and fault types, as shownin Table I.

TABLE I
Data set details (simulated from C-MAPSS)

Data Set	FD001	FD002	FD003	FD004
Train trajectories	100	260	100	249
Test trajectories	100	259	100	248
Conditions	1	6	1	6
Fault modes	1	1	2	2

The data set is further divided into training and test subsets which consist of multiple multivariate time series. Each time series is from a different engine, and the data can be considered to be from a fleet of engines of the same type. There are three operational settings that have a substantial effect on engine performance. These settings along with 21-dimensional sensor information are included in the data.

The engine starts with different degrees of initial wear and manufacturing variation which is unknown to the user. This wear and variation are considered normal, instead of a fault condition. The data is contaminated with sensor noise. The engine is operating normally at the start of each time series and develops a fault at some point during the series. In the training set, the fault grows in magnitude until system failure. In the test set, the time series ends sometime prior to system failure, and the true RUL value of the truncation point is given in a separate file. Our objective is to predict the number of remaining operational cycles before failure in the test set.

B. Experiment setup

1) Model and data design: Since the learning ability of the model is affected by the parameters to a certain extent, it is important to set appropriate parameters during model training. The main parameters involved in the network model are BayesianTCN structure parameters, and hyperparameters during training, etc. The parameter settings are shown in Table II.

TABLE II
Model parameter settings

Model	Parameter	Quantity
BayesianTCN	number of blocks	4
	dilations in each block	[1,2,4,8]
	kernel size	2
	out channels	1
hyperparameters	epoch	4
	learning rate	0.0001
	sample times	3

For input data, the dimension of feature vectors is a definite value. And for batch size and sliding window size, through setting different size combinations and comparing the results of experiments, a group of more appropriate parameters are obtained. In all of the following experiments, we set the sliding time window length to 32 and the batch size to 8.

2) Evaluation metrics: Since our prediction result is not a definite numerical point, but a confidence interval containing multiple prediction values, the prediction result can be measured from two aspects as follows.

- *Accuracy assessment*: For the input set, we apply 5 regression indicators in the Scikit-learn library: explained variance, MAE, MSE, MedAE, R2 score to evaluate the difference between the predicted value (denoted as Y_{mean} in the algorithm) and the label value.
- *Uncertainty measures*: In this experiment, the standard deviation of the posterior distribution of the model output is obtained as the uncertainty measurement index. Besides, for the obtained confidence interval CI, we can further evaluate the accuracy of the prediction by checking whether the interval contains the label value.

C. Experimental results

In this section, we first make point estimation on the FD001 data set to verify the effectiveness of the model proposed, and state two modeling methods for sequence prediction problem. Then we show the results of uncertainty quantification, namely, interval estimation of the model. Finally, we compare our model with other common models on all data sets from C-MAPSS and evaluate them using the metrics mentioned in Section III-B.

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.

1) Point Estimation on Test Sets: The test set of FD001 takes last time window data before the truncation point of each engine, and we predict the RUL at that point. Fig.3 shows the predicted curves on BayesianTCN, TCN, BayesianLSTM, and LSTM. The black curve represents the true value of RUL. The horizontal axis indicates the engine number. Correspondingly, Table III shows the evaluation results of the four networks.

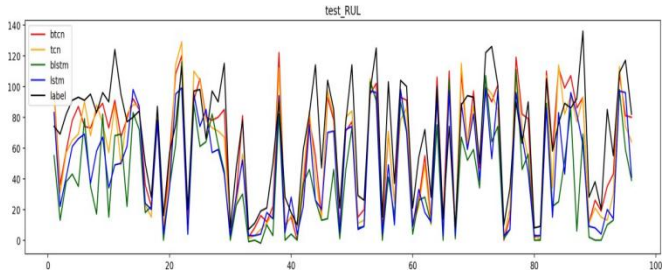


Fig. 3. Sequence-to-One fitting curve

TABLE III
EVALUATION RESULTS OF FOUR MODELS ON TEST DATASETS

	Explained Variance	MSE	MAE	MedAE	R2
LSTM	0.608064	923.5432	23.616804	19.329987	0.26912
BLSTM	0.6874	865.3088	23.6313	19.9330	0.3152
TCN	0.68975	536.49994	18.484694	15.891095	0.57542
BTCN	0.75496	372.43274	14.827587	10.836342	0.70526

As can be seen from the figure, the red and yellow curves are closer to the true value curve than the blue and green ones, indicating that networks based on TCN structures have higher accuracy in time series data prediction. In RNN structure, the gradient will accumulate along the time dimension at back propagation, making the network hard to optimize in the later stage of training due to gradient vanishing. While the convolution structure in TCN avoids such problem, thus has better performance. Moreover, among the four curves, the red one fits the true value curve best due to the narrowest distance, which proves the superiority of BayesianTCN in predicting remaining useful life. Data in Table III proves the same prediction effect as Fig. 3. In terms of MSE, MAE, and MedAE, our network has smaller values, and its evaluation results on explained variance and R-square is larger. Therefore, BayesianTCN not only fits the real RUL curve well but also makes the independent variable well explain the variance change of the dependent variable.

2) Two Time series Modeling Cases: For any engine in data sets, we have two modeling cases to obtain the RUL fitting curve from operation to failure. Given the whole run-to-failure feature sequence, the two cases have different label setting ways illustrated as below.

- *Sequence-to-One Case*: This case predicts the RUL at the last moment in a time window, as shown in Fig. 4. For the input sequence $\mathbf{x} = (x_1, \ldots, x_T)$, we slide along the time dimension with w window size. At the i-th slide, we get the input $(x_{i+1}, \ldots, x_{i+w})$ and the corresponding label y_{i+w}. In this case, after concatenating the outputs of each sliding window, we can get a complete RUL sequence $(y_w, \ldots, y_T)$.
- *Sequence-to-Sequence Case*: This case predicts the RUL at each moment in a time window. For each engine, we respectively divide its whole feature sequence and RUL sequence into segments with length w, and thus obtain $k(k = T/w)$ groups of input and output sub-sequence, as shown in Fig. 4. Therefore, the connection of the outputs makes up a complete RUL sequence starting at moment $s(s = T, \ldots, w)$.

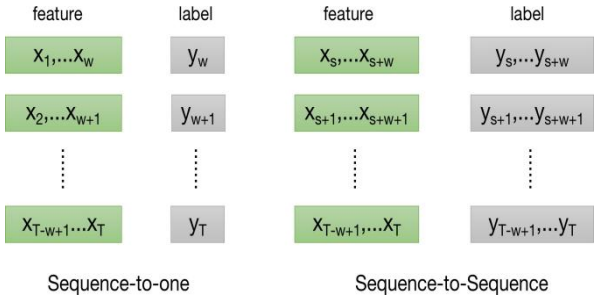


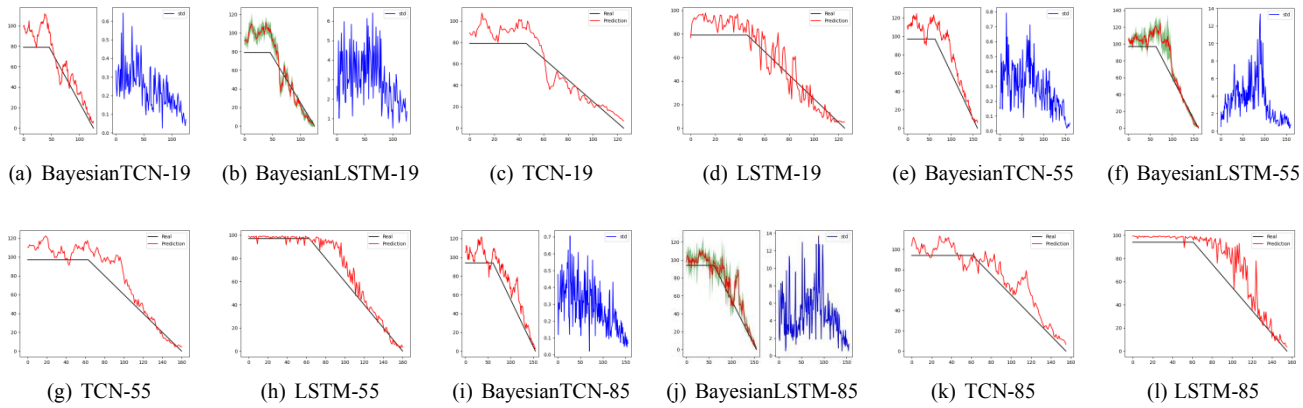
Fig. 4. sequence-to-one and sequence-to-one data flow

For each case, we randomly select 3 engines from 100 engines of FD001 to display, shown in Fig. 5 and Fig.6. In the four subfigures of an engine, the red curve represents the predicted RUL and the black curve represents the true RUL, and the corresponding uncertainty trend (measured by standard deviation) of BayesianTCN and BayesianLSTM is on the right side of first two subfigures. The operating cycle curves of the other machines are roughly the same. From the fitting degree of prediction curve and real curve, the two modeling cases can well predict the remaining life, indicating their effectiveness respectively, although sequence-to-sequence case gives a relatively wider confidence interval under the same confidence conditions compared to the other.

Networks based on the same structure (TCN or LSTM) are similar in curve shape, which shows that the introduction of the Bayesian method has no great influence on the prediction results of the original network. Moreover, BayesianTCN performs better than BayesianLSTM, partially because of the expanded convolution that enlarges the receptive field so that each convolution acquires information in a larger time range. However, on the fitting curve of some engines (e.g. No.19), BayesianTCN does not perform better than TCN at some points, which is reasonable that outliers may appear when sampling and affect the mean value. Comparing blue curves in subfigure a and b, the uncertainty of BayesianTCN is around one-tenth of that of BayesianLSTM, which means that BayesianTCN generates more stable prediction results for the same input data.

3) Uncertainty Quantification: Since the RUL of the machine is affected by many factors, the prediction has a relatively large uncertainty. Compared with point estimation,

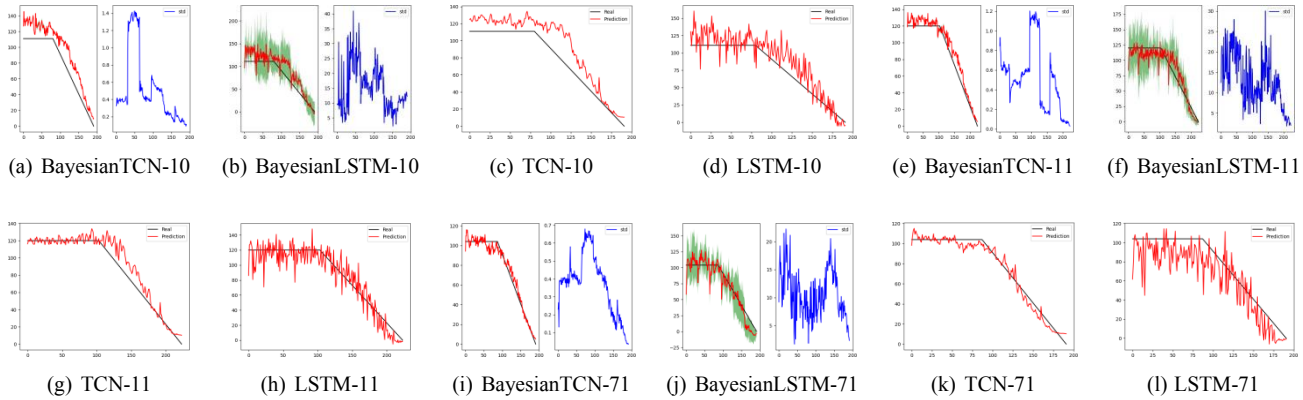
Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.



(a) BayesianTCN-19 (b) BayesianLSTM-19 (c) TCN-19 (d) LSTM-19 (e) BayesianTCN-55 (f) BayesianLSTM-55

(g) TCN-55 (h) LSTM-55 (i) BayesianTCN-85 (j) BayesianLSTM-85 (k) TCN-85 (l) LSTM-85

Fig. 5. Sequence-to-One whole life cycle of engine 19,55,85



(a) BayesianTCN-10 (b) BayesianLSTM-10 (c) TCN-10 (d) LSTM-10 (e) BayesianTCN-11 (f) BayesianLSTM-11

(g) TCN-11 (h) LSTM-11 (i) BayesianTCN-71 (j) BayesianLSTM-71 (k) TCN-71 (l) LSTM-71

Fig. 6. Sequence-to-Sequence whole life cycle of engine 10,11,71

giving a confidence interval can provide better maintenance decision support. Here we set confidence level $Z_{\alpha/2} = 20$, then according to Eq. (11) the interval estimation of RUL is expressed as $[\ Y_{mean} - 20 * Y_{std}\ ,\ \ Y_{mean} + 20 * Y_{std}\ .]$, shown by the green area in Fig. 7.

In most time periods, the predication curve fits well with the label curve. Besides, the confidence interval provides a higher error tolerance rate, thus even when the deviation between the predicted value and the real value is relatively large, the upper or lower bound of the interval can come closer to the label value. In this case, BayesianTCN can always manage to present a more stable output by adjusting the prediction results with a confidence interval, which is a significant advantage over TCN.

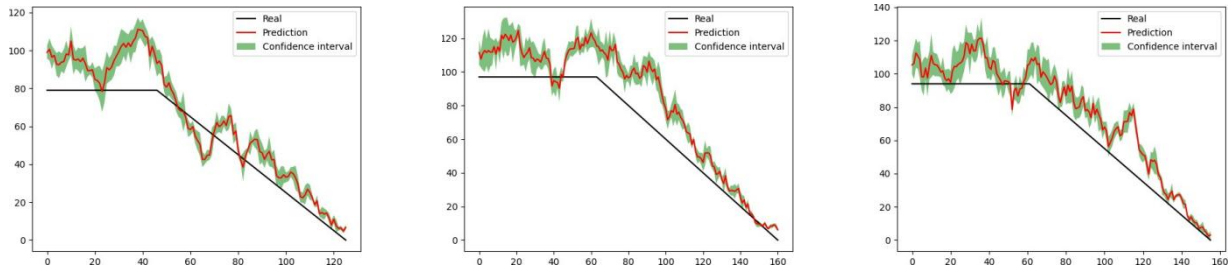
In the scenario of real remaining life prediction, we can observe whether the machine is running stably by observing the uncertainty curve, and at the same time combine it with the output confidence interval, so that we can provide more reliable prediction results.

4) Comparison: The RUL prediction of BayesianLSTM, CNN+BayesianLSTM, TCN+BayesianLSTM, BayesianTCN+BayesianLSTM, and two application modes of BayesianTCN (denoted as BTCN1 and BTCN2) on the four data sets FD001, FD002, FD003, and FD004 of C-MAPSS are performed in sequence to one method. Parameters in the same network are set the same in each model, and all have reached the fit. We use evaluation metrics mentioned in Section III-B to evaluate their prediction results, as shown in Table IV.

BTCN1 is the method we propose in this paper. For the input matrix of size $T \times k$, it extracts features along sequence dimension and outputs the vector $c = (c_1, \ldots, c_T)$. While in BTCN2, the kernel of conv1d layer moves along the feature dimension, and the output is $f = (f_1, \ldots, f_k)$, thus the eigenvalues of each type of information within the time window is acquired. In CNN + BayesianLSTM model, 1D-convolution kernel in CNN moves along time series, and after linear integration, we can obtain $T \times r$ matrix (r represents the reduced feature dimension) for BayesianLSTM to get RUL.

As shown in the table, BTCN1 performs best on FD001 and FD003, which are measured in relatively simple scenarios since the engines have fewer kinds of operation modes and failure modes. As for data sets FD002 and FD004 which are acquired in relatively complex scenes, BayesianTCN+BayesianLSTM and BTCN2 are respectively superior to other models in most evaluation metrics. In conclusion, the introduction of BayesianTCN structure in models can make it perform better in scenes whether it's complex or simple. Thanks to its ability to quantify uncertainty and the output of confidence interval, it can provide strong support for

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.



(a) BayesianTCN-19 (b) BayesianTCN-55 (c) BayesianTCN-85

Fig. 7. Whole life cycle of engine 19,55,85 with confidence intervel

TABLE IV
EVALUATION RESULTS OF THE SIX MODELS UNDER EACH DATA SET

		explained variance	MSE	MAE	MedAE	R2
FD001	BLSTM	0.6874	865.3088	23.6313	19.9330	0.3152
	CNN+BLSTM	0.7231	509.1501	17.8801	14.4379	0.5971
	TCN+BLSTM	**0.7578**	465.1569	17.1545	15.2444	0.6319
	BTCN+BLSTM	0.6588	818.5667	22.9696	18.6442	0.3522
	BTCN1	0.7188	**409.9645**	**15.5384**	**13.0210**	**0.6756**
	BTCN2	0.6164	727.4835	20.7822	17.0644	0.4243
FD002	BLSTM	0.7113	501.8177	16.3704	11.9677	0.6846
	CNN+BLSTM	0.7296	540.8120	16.4021	**11.0144**	0.6601
	TCN+BLSTM	**0.7502**	479.5229	16.5955	12.1545	0.6986
	BTCN+BLSTM	0.7225	**448.5276**	**15.7209**	11.7442	**0.7181**
	BTCN1	0.5823	727.4947	20.2858	15.4796	0.5427
	BTCN2	0.6789	511.84885	17.5695	14.4899	0.6783
FD003	BLSTM	0.4454	902.6858	24.4811	23.0000	0.2841
	CNN+BLSTM	0.4633	932.4942	25.1040	22.6093	0.2604
	TCN+BLSTM	0.5048	792.9575	24.2314	23.6150	0.3711
	BTCN+BLSTM	0.4636	681.1336	20.6695	16.7337	0.4598
	BTCN1	**0.5920**	**538.7500**	**18.5535**	**15.5204**	**0.5727**
	BTCN2	0.3560	838.2391	21.9151	17.2437	0.3352
FD004	BLSTM	0.5984	833.4446	21.7229	15.6361	0.5927
	CNN+BLSTM	0.5856	998.6503	22.8080	14.6105	0.5119
	TCN+BLSTM	**0.6264**	861.8150	21.3569	**13.8081**	0.5788
	BTCN+BLSTM	0.5004	1035.2478	24.8147	20.1575	0.4941
	BTCN1	0.4723	1258.2548	27.4645	22.2943	0.3851
	BTCN2	0.6114	**801.4501**	**21.2142**	14.8547	**0.6083**

the prediction of remaining useful life.

IV. CONCLUSION

RUL prediction is an important part of PHM, which is of significance to the safety of production and the reduction of financial loss. In this paper, a new structure named BayesianTCN is proposed to predict the RUL over sequence data and calculate the uncertainty of prediction . Apart from that, both point estimation and range estimation can be provided by BayesianTCN. BayesianTCN takes advantages of TCN: flexible receptive field size, a faster and more stable converging performance compared with Bayesian-LSTM structure which is also designed to solve above problems. The experiments on the CMAPSS data set show that BayesianTCN has higher accuracy and the uncertainty is significantly reduced in most cases, making the prediction results more reliable and contributing to the predictive maintenance strategies.

REFERENCES

[1] Kwok Chui, B B Gupta, and Pandian Vasant. A genetic algorithm optimized rnn-lstm model for remaining useful life prediction of turbofan engine. *Electronics*, 10:285, 01 2021.

[2] Chang Hua Hu, Hong Pei, Xiao Sheng Si, Dang Bo Du, and Xi Wang. A prognostic model based on dbn and diffusion process for degrading bearing. *IEEE Transactions on Industrial Electronics*, PP(99):1–1, 2019.

[3] D. Huang, R. Bai, S. Zhao, P. Wen, and S. Chen. Bayesian neural network based method of remaining useful life prediction and uncertainty quantification for aircraft engine. In *2020 IEEE International Conference on Prognostics and Health Management (ICPHM)*, 2020.

[4] Yaguo Lei, Naipeng Li, Liang Guo, Ningbo Li, Tao Yan, and Jing Lin. Machinery health prognostics: A systematic review from data acquisition to rul prediction. *Mechanical Systems and Signal Processing*, 104(MAY1):799–834,

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.

2018.

[5] C. Liu, L. Zhang, and C. Wu. Direct remaining useful life prediction for rolling bearing using temporal convolutional networks. In *2019 IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 2965–2971, 2019.

[6] Meng Ma and Zhu Mao. Deep convolution-based lstm network for remaining useful life prediction. *IEEE Transactions on Industrial Informatics*, PP(99):1–1, 2020.

[7] Weiwen Peng, Zhi Sheng Ye, and Nan Chen. Bayesian deep learning based health prognostics towards prognostics uncertainty. *IEEE Transactions on Industrial Electronics*, pages 1–1, 2019.

[8] Daoming She and Minping Jia. A bigru method for remaining useful life prediction of machinery. *Measurement*, 167:108277, 2020.

[9] Xiaoyang Zhang, Xin Lu, Weidong Li, and Sheng Wang. Prediction of the remaining useful life of cutting tool using the hurst exponent and cnn-lstm. *The International Journal of Advanced Manufacturing Technology*, (2), 2021.

Authorized licensed use limited to: University at Buffalo Libraries. Downloaded on May 27,2024 at 01:32:27 UTC from IEEE Xplore. Restrictions apply.